

```

        c = game.makeMoveCommand(NORTH);
    else if (command.equalsIgnoreCase("s"))
        c = game.makeMoveCommand(SOUTH);
    else if (command.equalsIgnoreCase("r"))
        c = game.makeRestCommand();
    else if (command.equalsIgnoreCase("sw"))
        c = game.makeShootCommand(WEST);
    else if (command.equalsIgnoreCase("se"))
        c = game.makeShootCommand(EAST);
    else if (command.equalsIgnoreCase("sn"))
        c = game.makeShootCommand(NORTH);
    else if (command.equalsIgnoreCase("ss"))
        c = game.makeShootCommand(SOUTH);
    else if (command.equalsIgnoreCase("q"))
        return;

    c.execute();
}
}

```

Notice also that `main` creates the input stream and contains the main loop of the game, interpreting the simple input commands, but then defers all processing to other, higher-level components.

Finally, notice that `main` creates the map.

[Click here to view code image](#)

```

private static void createMap() {
    int nCaverns = (int) (Math.random() * 30.0 + 10.0);
    while (nCaverns-- > 0)
        caverns.add(makeName());

    for (String cavern : caverns) {
        maybeConnectCavern(cavern, NORTH);
        maybeConnectCavern(cavern, SOUTH);
        maybeConnectCavern(cavern, EAST);
        maybeConnectCavern(cavern, WEST);
    }

    String playerCavern = anyCavern();
    game.setPlayerCavern(playerCavern);
    game.setWumpusCavern(anyOther(playerCavern));
    game.addBatCavern(anyOther(playerCavern));
    game.addBatCavern(anyOther(playerCavern));
    game.addBatCavern(anyOther(playerCavern));
}

```

```
game.addPitCavern(anyOther(playerCavern));  
game.addPitCavern(anyOther(playerCavern));  
game.addPitCavern(anyOther(playerCavern));  
  
game.setQuiver(5);  
}  
  
// much code removed...  
}
```

The point is that `Main` is a dirty low-level module in the outermost circle of the clean architecture. It loads everything up for the high level system, and then hands control over to it.

CONCLUSION

Think of `Main` as a plugin to the application—a plugin that sets up the initial conditions and configurations, gathers all the outside resources, and then hands control over to the high-level policy of the application. Since it is a plugin, it is possible to have many `Main` components, one for each configuration of your application.

For example, you could have a `Main` plugin for *Dev*, another for *Test*, and yet another for *Production*. You could also have a `Main` plugin for each country you deploy to, or each jurisdiction, or each customer.

When you think about `Main` as a plugin component, sitting behind an architectural boundary, the problem of configuration becomes a lot easier to solve.

SERVICES: GREAT AND SMALL



Service-oriented “architectures” and micro-service “architectures” have become very popular of late. The reasons for their current popularity include the following:

- Services seem to be strongly decoupled from each other. As we shall see, this is only partially true.
- Services appear to support independence of development and deployment. Again, as we shall see, this is only partially true.

SERVICE ARCHITECTURE?

First, let’s consider the notion that using services, by their nature, is an architecture. This is patently untrue. The architecture of a system is defined by boundaries that

separate high-level policy from low-level detail and follow the Dependency Rule. Services that simply separate application behaviors are little more than expensive function calls, and are not necessarily architecturally significant.

This is not to say that all services *should* be architecturally significant. There are often substantial benefits to creating services that separate functionality across processes and platforms—whether they obey the Dependency Rule or not. It's just that services, in and of themselves, do not define an architecture.

A helpful analogy is the organization of functions. The architecture of a monolithic or component-based system is defined by certain function calls that cross architectural boundaries and follow the Dependency Rule. Many other functions in those systems, however, simply separate one behavior from another and are not architecturally significant.

So it is with services. Services are, after all, just function calls across process and/or platform boundaries. Some of those services are architecturally significant, and some aren't. Our interest, in this chapter, is with the former.

SERVICE BENEFITS?

The question mark in the preceding heading indicates that this section is going to challenge the current popular orthodoxy of service architecture. Let's tackle the benefits one at a time.

THE DECOUPLING FALLACY

One of the big supposed benefits of breaking a system up into services is that services are strongly decoupled from each other. After all, each service runs in a different process, or even a different processor; therefore those services do not have access to each other's variables. What's more, the interface of each service must be well defined.

There is certainly some truth to this—but not very much truth. Yes, services are decoupled at the level of individual variables. However, they can still be coupled by shared resources within a processor, or on the network. What's more, they are strongly coupled by the data they share.

For example, if a new field is added to a data record that is passed between services,

then every service that operates on the new field must be changed. The services must also strongly agree about the interpretation of the data in that field. Thus those services are strongly coupled to the data record and, therefore, indirectly coupled to each other.

As for interfaces being well defined, that's certainly true—but it is no less true for functions. Service interfaces are no more formal, no more rigorous, and no better defined than function interfaces. Clearly, then, this benefit is something of an illusion.

THE FALLACY OF INDEPENDENT DEVELOPMENT AND DEPLOYMENT

Another of the supposed benefits of services is that they can be owned and operated by a dedicated team. That team can be responsible for writing, maintaining, and operating the service as part of a dev-ops strategy. This independence of development and deployment is presumed to be *scalable*. It is believed that large enterprise systems can be created from dozens, hundreds, or even thousands of independently developable and deployable services. Development, maintenance, and operation of the system can be partitioned between a similar number of independent teams.

There is some truth to this belief—but only some. First, history has shown that large enterprise systems can be built from monoliths and component-based systems as well as service-based systems. Thus services are not the only option for building scalable systems.

Second, the decoupling fallacy means that services cannot always be independently developed, deployed, and operated. To the extent that they are coupled by data or behavior, the development, deployment, and operation must be coordinated.

THE KITTY PROBLEM

As an example of these two fallacies, let's look at our taxi aggregator system again. Remember, this system knows about many taxi providers in a given city, and allows customers to order rides. Let's assume that the customers select taxis based on a number of criteria, such as pickup time, cost, luxury, and driver experience.

We wanted our system to be scalable, so we chose to build it out of lots of little micro-services. We subdivided our development staff into many small teams, each of which is responsible for developing, maintaining, and operating a correspondingly¹ small number of services.

The diagram in [Figure 27.1](#) shows how our fictitious architects arranged services to implement this application. The `TaxiUI` service deals with the customers, who use mobile devices to order taxis. The `TaxiFinder` service examines the inventories of the various `TaxiSuppliers` and determines which taxis are possible candidates for the user. It deposits these into a short-term data record attached to that user. The `TaxiSelector` service takes the user's criteria of cost, time, luxury, and so forth, and chooses an appropriate taxi from among the candidates. It hands that taxi off to the `TaxiDispatcher` service, which orders the appropriate taxi.

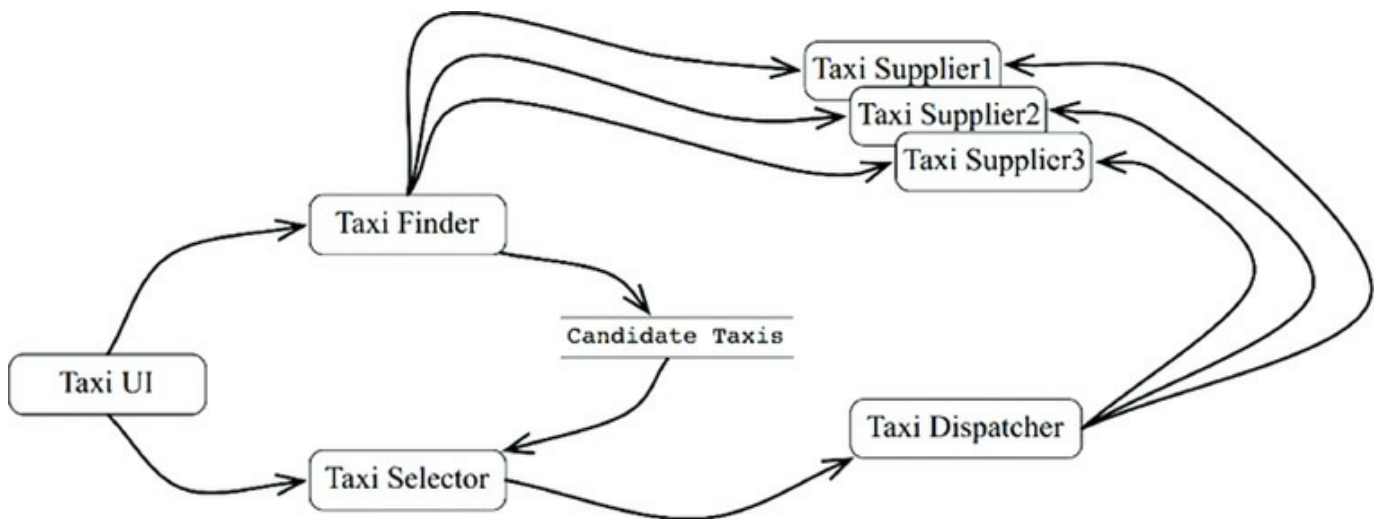


Figure 27.1 Services arranged to implement the taxi aggregator system

Now let us suppose that this system has been in operation for more than a year. Our staff of developers have been happily developing new features while maintaining and operating all these services.

One bright and cheerful day, the marketing department holds a meeting with the development team. In this meeting, they announce their plans to offer a kitten delivery service to the city. Users can order kittens to be delivered to their homes or to their places of business.

The company will set up several kitten collection points across the city. When a kitten order is placed, a nearby taxi will be selected to collect a kitten from one of those collection points, and then deliver it to the appropriate address.